Critic-/Value-based RL for Diffusion & Flow-Matching Models

A Comparative Analysis of Four Papers and the Flow-Factory PPO Trainer

Flow-Factory

June 19, 2026

Abstract

This note compares four recent papers on RL fine-tuning of diffusion / flow-matching generators against Flow-Factory's critic-based PPO trainer (`trainer_type: ppo`). All five methods share a single thesis: replace GRPO's coarse, trajectory-level, *group-relative* advantage with a learned, timestep-conditioned *state value function* $V_\phi(\boldsymbol{z}_t, t)$ on noisy latents for fine-grained credit assignment. They diverge along two orthogonal axes: (i) **how the value becomes a parameter update** (the actor mechanism) and (ii) **what network realizes the value** (the critic body). The papers are: *DiffAC* [1] (arXiv:2403.04154), *VARD* [2] (arXiv:2505.15791), *AC-Flow* [3] (arXiv:2510.18072), and *Explicit Critic Guidance* [4] (arXiv:2605.27736). A central, easy-to-miss finding: *only* Explicit Critic Guidance is also *true* PPO (clipped policy-gradient surrogate + GAE), like ours; the other three keep a state-value critic but replace the PPO actor with, respectively, a one-step actor–critic on forward-perturbed states, a differentiable value back-propagated through the sampler, and an advantage-weighted flow-matching regression. For each paper we state its unique contribution, its core equations, and—concretely—how it would be implemented in the Flow-Factory codebase (what already exists, what is new). The companion note `docs/ppo/ppo_analysis` documents our trainer in depth.

Contents

1 Scope and the One Shared Idea

Flow-Factory fine-tunes flow-matching / diffusion models with online RL against (typically non-differentiable) reward models. The default algorithm, GRPO, treats a whole denoising rollout as one "action": it computes a single scalar advantage per sample by *group-relative* reward normalization and broadcasts it to every denoising step. This is simple and strong but gives no per-step credit and uses a group-mean (state-blind) baseline.

Every method in this note attacks exactly that limitation with the *same* core device:

> *Learn a timestep-conditioned value $V_\phi(\boldsymbol{z}_t, t)$ on the noisy latent state that the policy actually visits, and use it for fine-grained, per-step credit assignment.*

Because the reward is terminal-only, the value at any intermediate state is the *expected final reward given that state*, $V_\phi(\boldsymbol{z}_t, t) \approx \mathbb{E}[R \mid \boldsymbol{z}_t]$, and its Monte-Carlo regression target is just the terminal reward R. (With $\gamma = \lambda = 1$ even GAE collapses to this Monte-Carlo target at every step.) That shared skeleton is what makes the five methods comparable; everything interesting is in *how each one turns V_ϕ into a gradient.*

The four papers at a glance. Table 1 fixes identities. Two are published/peer-track (DiffAC, ICML 2024) and three are preprints under review. They span UNet, DiT/MMDiT and SE(3) backbones, and image / molecule / protein tasks, but the recurring testbed is text-to-image LoRA fine-tuning, which is also our primary use case.

Short name	Title (abbrev.) / venue	Backbone(s)	Task / reward
DiffAC arXiv:2403.04154	Stabilizing Policy Gradients for SDEs via Consistency with Perturbation Process; *ICML 2024*	TargetDiff (SE(3) diff.); SD1.5 (UNet, LoRA)	Drug design (Vina); T2I (ImageReward)
VARD arXiv:2505.15791	VARD: Efficient and Dense Fine-Tuning with Value-based RL; *preprint*	SD1.4 / SDXL (UNet); FoldFlow-2 (flow)	T2I (Aesthetic, PickScore, ImageReward, HPSv2; compressibility); protein
AC-Flow arXiv:2510.18072	Fine-tuning Flow Matching Generative Models with Intermediate Feedback; *preprint*	SD3 (MMDiT, LoRA)	T2I on DrawBench (CLIP score)
Explicit Critic arXiv:2605.27736	Explicit Critic Guidance for Aligning Diffusion Models; *preprint*	SD1.5 (UNet); SD3.5M (DiT)	T2I (CLIP, HPSv2.1, PickScore, GenEval, OCR)
Flow-Factory PPO (ours)	Critic-based PPO (`trainer_type: ppo`); see `ppo_analysis`	Model-agnostic (conv / packed / video)	Any registered reward (terminal weighted sum)

Table 1: The four papers and our trainer. "Explicit Critic" is the paper our team previously referenced; it is the only one besides ours that is genuine PPO.

A naming caveat worth stating up front. "Actor–critic", "value", "advantage", and "critic warmup" appear in all four papers, but three of them are *not* PPO. Section 2 makes the distinction precise; without it the comparison is misleading.

2 A Unifying MDP and the Two Axes of Variation

2.1 Denoising as a finite-horizon MDP

All methods model the ordered sampling steps as an MDP (following DDPO/Flow-GRPO): state $s_t = (\boldsymbol{z}_t, t)$ (a noisy latent plus its timestep; some add the text condition y), action $a_t =$ the next latent $\boldsymbol{z}_{t-1}$ (equivalently the predicted denoising direction), the policy is the per-step transition, and the reward is *terminal only*, $R = \sum_k w_k\, r_k(\text{decode}(\boldsymbol{z}_0))$. The value $V_\phi(\boldsymbol{z}_t, t)$ estimates the expected terminal reward from $\boldsymbol{z}_t$, with terminal value 0. This is exactly the formulation in our `ppo_analysis` note.

2.2 Axis 1 — how V_ϕ becomes a gradient (the actor mechanism)

This is the dominant axis. There are four distinct mechanisms across the five methods:

1. **PPO clipped policy gradient** (*Ours*, *Explicit Critic*). V_ϕ is a *baseline*; GAE turns TD residuals into per-step advantages $\hat{A}_t$; the policy is updated by the clipped importance-ratio surrogate. The critic is never differentiated *into* the policy.

2. **One-step actor–critic on perturbed states** (*DiffAC*). REINFORCE with the *next* state's value as the return ($\nabla \log \pi \cdot V_\phi(\text{next})$), or DDPG (pathwise $\nabla_\theta V_\phi$). States are synthesized by *noising clean samples forward*, not by rollout.

3. **Differentiable value back-propagation** (*VARD*). The (reparam.) sampler is differentiable, so maximize $V_\phi(\boldsymbol{z}_t)$ directly by back-propagating its gradient *through* the sampling step into θ (deterministic policy gradient / reward-backprop). No log-probs, no ratio.

4. **Advantage-weighted regression** (*AC-Flow*). Sidestep the intractable flow-matching likelihood: re-weight the conditional flow-matching (CFM) regression loss by $\exp(\tau \hat{A}_t)$ (an AWR/RWR generalization). No log-probs, no ratio.

   Figure 1 draws the common trunk and the four branches.

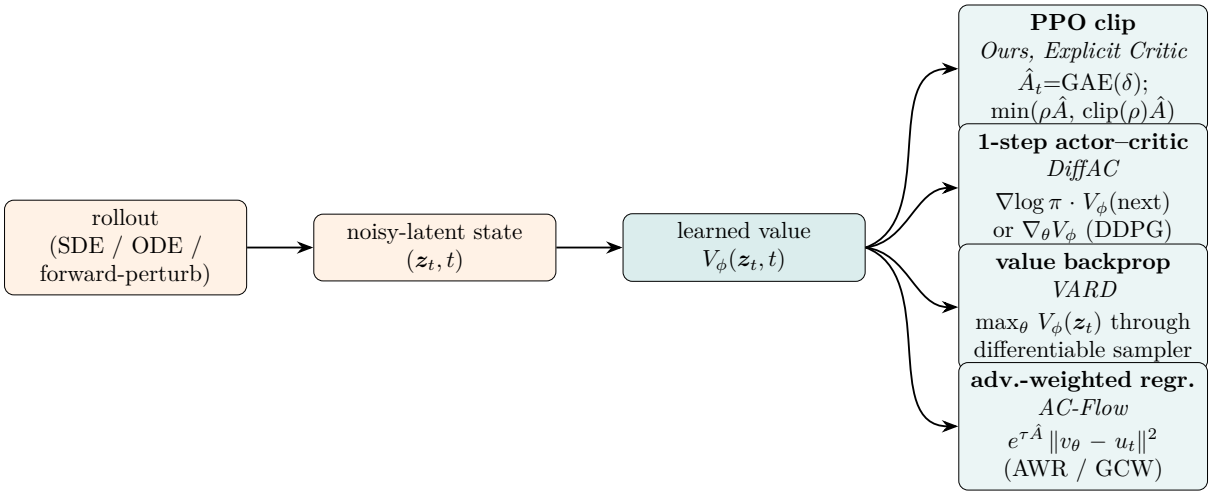

Figure 1: One shared value $V_\phi(\boldsymbol{z}_t, t)$, four ways to turn it into a parameter update. Only the top branch (ours and Explicit Critic) is PPO.

2.3 Axis 2 — what network is the critic (the critic body)

- **Independent lightweight net**: *Ours* (token-fold + residual MLP + learned-query attention pooling + sinusoidal time), *AC-Flow* (CNN+MLP), *DiffAC* (backbone-shaped for molecules; T2I head unspecified).
- **The diffusion backbone itself**: *Explicit Critic* (value heads on the shared backbone; the policy *is* the critic body).
- **The reward model**: *VARD* (LoRA-adapt the reward model into the value net; a plain MLP head was reported insufficient for preference rewards).

The two axes are independent: ours and Explicit Critic share Axis 1 (PPO) but sit at opposite ends of Axis 2 (lightweight independent net vs. backbone-as-critic). This 2× split is the backbone of the per-paper analysis below.

3 Per-paper Contribution and Flow-Factory Implementation

Each subsection gives (a) the *unique* contribution, (b) the core method/equations, (c) the relation to our trainer, and (d) a concrete implementation mapping to the codebase (`trainers/ppo/{trainer,gae,hparams/training_args/ppo.py`).

3.1 DiffAC — consistency with the perturbation process (arXiv:2403.04154)

Unique contribution.

1. **Forward-perturbation (off-policy) gradient estimation.** Instead of estimating the policy gradient on sparse *backward* rollouts, DiffAC *noises the generated clean samples* $\boldsymbol{x}_0$ *forward* to synthesize an unlimited, space-covering set of intermediate states $\boldsymbol{x}_t \sim p_{t0}(\cdot \mid \boldsymbol{x}_0)$. Under a *consistency* condition (Lemma 4.2) these perturbed states follow the true rollout marginal, so the gradient is well-defined and low-variance even in high dimensions.
2. **Consistency enforced by a score-matching–refit reference + KL** (Eqs. 12–13), with a formal error bound (Thm 4.5: gradient error $\leq$ sum of per-step $\sqrt{\mathrm{KL}}$ between the consistent and the policy SDE).
3. **A value critic trained by direct Monte-Carlo regression to the terminal reward** (Eq. 8), and a single framework instantiable with either a REINFORCE or a *DDPG/pathwise* actor.

Core method. Critic target (with $\boldsymbol{x}_t$ a *forward-perturbed* state):

$$\phi^\star = \arg\min_\phi \mathbb{E}_{t,\, \boldsymbol{x}_0\sim q_0,\, \boldsymbol{x}_t\sim p_{t0}(\cdot|\boldsymbol{x}_0)} \big(V_\phi(\boldsymbol{x}_t,t) - R(\boldsymbol{x}_0)\big)^2, \qquad V^\star_\phi(\boldsymbol{x}_t,t) = \mathbb{E}_{\boldsymbol{x}_0\sim q_{0t}(\cdot|\boldsymbol{x}_t)}[R(\boldsymbol{x}_0)]. \quad (1)$$

Actor (one SDE step $\bar{\boldsymbol{x}}_{t-\mathrm{d}t}$ from a perturbed $\boldsymbol{x}_t$):

$$\text{SDE-REINFORCE (Eq. 10):} \quad \mathrm{PG}(\theta) = \tfrac{1}{n}\textstyle\sum_j \nabla_\theta \log P_\theta(\bar{\boldsymbol{x}}^j_{t-\mathrm{d}t} \mid \boldsymbol{x}^j_t)\, V_\phi(\bar{\boldsymbol{x}}^j_{t-\mathrm{d}t},\, t-\mathrm{d}t), \quad (2)$$

$$\text{SDE-DDPG (Eq. 11):} \quad \mathrm{PG}(\theta) = \tfrac{1}{n}\textstyle\sum_j \nabla_\theta V_\phi(\bar{\boldsymbol{x}}^j_t,\, t-\mathrm{d}t), \quad (3)$$

$$\text{Update (Eq. 13):} \quad \theta \leftarrow \theta + \eta_1\, \mathrm{PG}(\theta) - \eta_1\eta_2\, \nabla_\theta D_{\mathrm{KL}}(\varepsilon_\theta, \varepsilon_{\theta'}, \mathcal{D}), \quad (4)$$

where $\varepsilon_{\theta'}$ is a reference *re-fit by score matching* on the latest data each iteration. There is **no GAE and no PPO clip**; the only trust region is the consistency KL.

Relation to ours. Shared: SDE-as-MDP, terminal-only reward, a $V(\boldsymbol{x}_t, t)$ critic, a KL-to-reference, and critic pretraining. Different: DiffAC trains both critic and actor on *forward-perturbed* states (we use on-policy rollout latents), uses a *re-fit* reference (ours is a fixed/EMA reference), offers a *pathwise* actor (we are strictly likelihood-ratio), and uses a *one-step* value signal (we use multi-step GAE). With $\gamma = \lambda = 1$ our GAE return equals DiffAC's MC critic target — but evaluated on rollout vs. perturbed states.

Implementation in Flow-Factory. DiffAC's *actor* is neither PPO-clip nor GAE, so a faithful port is a **sibling trainer** (e.g. `trainers/diffac/`) reusing our infrastructure, not an edit to the PPO loss. Two scoped options:

- **Option A (faithful).** *New*: a forward-perturbation state sampler ($\boldsymbol{x}_t =$ `scheduler.add_noise`$(\boldsymbol{x}_0$, noise, needs a forward-marginal helper on the adapter); a one-step actor loss (REINFORCE $-\mathbb{E}[\log P_\theta(\bar{\boldsymbol{x}} \mid \boldsymbol{x}_t)\, V_\phi(\bar{\boldsymbol{x}}).\text{detach}()]$ or DDPG $-\mathbb{E}[V_\phi(\bar{\boldsymbol{x}})]$); a per-iteration score-matching re-fit of the reference. The critic loss becomes plain MSE to $R(\boldsymbol{x}_0)$ (drop `value_clip_range`, GAE, whitening). *Reused*: `generate_samples` rollout + per-step log-prob, `_compute_terminal_rewards`, dual optimizers, critic-warmup scaffolding, checkpointing, `_compute_kl` plumbing (extended to refit the reference).
- **Option B (cheap, additive — recommended follow-up).** Keep our PPO+GAE actor, add a `critic_target: {gae, perturb_terminal}` switch. In `perturb_terminal` mode, regress the critic on forward-perturbed states to $R(\boldsymbol{x}_0)$ (DiffAC Eq. 9); GAE then bootstraps off a better-calibrated, lower-variance value. A self-contained change to `_compute_old_values` / `_critic_value_loss` / `_fit_critic_only` plus a perturbation sampler and one hparam. The forward-noising itself is *already implemented* in `trainers/awm.py` (`noised_latents = (1 - sigma) * clean_latents + sigma * noise`), so the change is small. This follow-up is written up in full—plan, hparams, and caveats (Monte-Carlo target only, timestep-scale alignment, consistency gap)—in `docs/ppo/ppo_analysis` § "Follow-up: DiffAC-style perturbed-state critic training".

New hparams (Option A): `actor_estimator: reinforce|ddpg`, `consistency_kl_coef` (η_2), `reference_refit_steps`, `perturb_t_distribution`, off-policy buffer toggle.

3.2 VARD — back-propagating a learned dense value (arXiv:2505.15791)

Unique contribution.

1. **A learned value as a differentiable surrogate reward that you back-propagate.** Reward-backprop methods (DRaFT/ReFL) need a *differentiable reward*; policy-gradient methods (DDPO/DPOK) tolerate non-differentiable rewards but are high-variance. VARD trains a value/PRM that is differentiable *even when the reward is not*, then back-props *the value* through the sampler — getting stable analytic gradients for non-differentiable rewards. This dissolves the "non-differentiable reward ↔ stable training" dilemma.
2. **Dense per-step supervision via a Monte-Carlo value** (Eqs. 7–8): the value predicts the expected final reward from any intermediate state, turning a sparse terminal reward into a dense signal *without* TD/GAE.
3. **MSE-as-KL (Lemma 1):** per-step output-MSE between fine-tuned and pretrained models is gradient-equivalent to the per-step Gaussian KL, a cheap anchor that resists reward hacking.
4. **Practical recipe:** LoRA-adapt the *reward model* into the value net; a DPOK-style plain MLP head was insufficient for preference rewards.

Core method.

$$\text{Value (Eq. 7):} \quad V_\phi(\boldsymbol{x}_{T-t}, c) = \mathbb{E}\big[\textstyle\sum_{t' \geq t} R \mid \boldsymbol{x}_{T-t'}\big] = \mathbb{E}[\text{final reward} \mid \boldsymbol{x}_{T-t}], \tag{5}$$

$$\text{Value pretraining (Eq. 8):} \quad J(\phi) = \mathbb{E}\big[(r(\boldsymbol{x}_0, c) - V_\phi(\boldsymbol{x}_{T-t}, c))^2\big] \quad \text{(Monte-Carlo)}, \tag{6}$$

$$\text{Policy (Eq. 9):} \quad J(\theta) = -\mathbb{E}\big[\, V_\phi(\boldsymbol{x}_{T-t}, c) + \eta \|\boldsymbol{x}_{T-t} - \boldsymbol{x}^0_{T-t}\|^2\big], \tag{7}$$

$$\text{MSE}\leftrightarrow\text{KL (Eq. 10):} \quad \nabla_\theta\, \mathbb{E}\|\boldsymbol{x}_{T-t} - \boldsymbol{x}^0_{T-t}\|^2 = 2\sigma^2\, \nabla_\theta D_{\mathrm{KL}}(p_\theta \| p_{\theta_0}). \tag{8}$$

The gradient flows through the differentiable value and the reparameterized sampler into θ. **No importance ratio, no PPO clip, no GAE**; requires a differentiable sampler. Images fine-tune only the final 10 of 50 steps.

Relation to ours. Strongly shared ingredients: MDP, terminal reward, a separate value net, the *Monte-Carlo* value target (our `compute_gae` with $\gamma = \lambda = 1$ gives exactly returns $\equiv R$ at every step), value pretraining + online refresh (our `critic_warmup_steps` + periodic re-warmup), and the *x-space KL anchor* (our `kl_type="x-based"`, $\eta \leftrightarrow$`kl_beta`). The crux difference: VARD's *policy update is value backprop through the sampler* (the value is the objective), whereas ours uses the value as a *detached baseline* in a score-function PPO gradient.

Implementation in Flow-Factory. Add a `policy_objective: {ppo-clip, value-backprop}` mode (or a `VARDTrainer`); it reuses ∼70% of our scaffolding but swaps the policy loss. *No trainer in the repo currently does differentiable-rollout reward backprop*, so this is genuinely new infrastructure.

- **Reused as-is:** value warmup/refresh (= VARD Stage 1 + online value fine-tune); the MC value target (γ=λ=1); the x-based KL/MSE anchor (`_compute_kl`, $\eta \leftrightarrow$`kl_beta`); the reference model (`use_ref_parameters`); last-k-step training (`train_timesteps` / `compute_trajectory_indice`
  The scheduler's `next_latents_mean` is already differentiable w.r.t. policy params.
- **Main change (`optimize`):** replace the PPO-clip block. Forward the policy step *with grad* (do not detach `next_latents`); compute V_ϕ(next) *with grad enabled*; loss = $-V_\phi$(next).mean()+ `kl_beta` · MSE. Remove the ratio/clip/advantage path. A per-step single-step backprop keeps memory bounded.
- **Critic body (optional, recommended):** a mode to use a LoRA-adapted reward model as the value net (our `ValueCritic` is the cheaper fallback they found weaker).
- **Hparams:** `policy_objective`; reuse `kl_beta` as η (note the paper sweeps $\eta \in [0.1, 100]$); in this mode, validate that ratio/GAE knobs are inert (fail-fast on misconfig).

Caveat: backprop through the backbone per step is more memory-hungry than our log-prob replay (paper mitigates via last-10-steps + LoRA + grad-accum); it needs a differentiable sampler (true for our flow scheduler's `next_latents_mean`).

3.3 AC-Flow — advantage-weighted flow-matching regression (arXiv:2510.18072)

Unique contribution.

1. **Generalized Critic Weighting (GCW).** The flow-matching likelihood is intractable ($\nabla \cdot v_\theta$ is #P-hard), so AC-Flow replaces the policy-gradient update with an $\exp(\tau \cdot \text{signal})$-weighted *CFM regression* that unifies RWR (Ψ=r), AWR/GRAE (Ψ=$(r-\mu)/\sigma$), and critic-advantage (Ψ=$r-V$) — and instantiates it with a per-state *critic advantage* for intermediate feedback.

2. **A dual-stability recipe** that makes online flow actor–critic converge where naive value regression diverges (actor loss $10^{12} \to 10^2$): *min–max reward shaping* (Eq. 6), *advantage clipping* ($\pm\delta$=5), and a *GRAE-weighted critic warm-up* (actor keeps training with group-relative weights while the critic matures, k=500).

3. **Wasserstein-2 (velocity-L^2) regularization** (Thm 3.3) in place of the intractable KL, with a cheap CNN+MLP critic ($\sim$2 GB / $\sim$2 h overhead).

Core method. Sampling is *deterministic ODE*; "intermediate states" are CFM interpolations $\boldsymbol{x}_t = t\,\boldsymbol{x}_1 + (1-t)\,\boldsymbol{x}_0$ with target $u_t = \boldsymbol{x}_1 - \boldsymbol{x}_0$.

$$\text{Reward shaping (Eq. 6):}\ \ \mathrm{RS}(r_i) = \frac{r_i - \min(r)}{\max(r) - \min(r) + \epsilon}, \tag{9}$$

$$\text{Critic (Eq. 10):}\ \ L_{\text{critic}} = \mathbb{E}\big[(V_\phi(\boldsymbol{x}_t, t, c) - \mathrm{RS}(r))^2\big], \qquad A = \mathrm{RS}(r) - V_\phi(\boldsymbol{x}_t, t, c), \tag{10}$$

$$\text{Actor (Eqs. 11–12):}\ \ L_{\text{actor}} = \mathbb{E}\big[e^{\tau\,\mathrm{clip}(A,\pm\delta)}\|v_\theta - u_t\|^2\big] + \alpha \int_0^1 \mathbb{E}\|v_\theta - v_{\theta_{\text{ref}}}\|^2\,\mathrm{d}s. \tag{11}$$

No importance ratio, no PPO clip, no GAE, no per-step log-probs; the advantage is one-sample Monte-Carlo $A = r - V$ (a "GAE-inspired" family is described but not used).

Relation to ours. Shared: a separate lightweight time-conditioned scalar critic; advantage clipping at ± 5 (our `adv_clip_range`); a velocity-L^2 reference penalty (our `kl_type="v-based"` = their W_2 surrogate); a critic warmup; separate backward passes. Different: AC-Flow's *actor is advantage-weighted CFM regression* (not a ratio loss), the advantage is *MC* not GAE, it uses *min–max reward shaping* (we whiten advantages instead), an *exp-weight temperature* τ, a *GRAE-weighted* warmup (we freeze the policy during bootstrap), and ODE sampling (no per-step log-prob storage).

Implementation in Flow-Factory. A new actor-loss mode (`policy_objective: gcw`) on `PPOTrainer`, or a sibling trainer (conceptually the reward-weighted/RWR flow trainer + a critic weight).

- **Reward (`prepare_feedback`):** add min–max reward shaping RS(r) over the global batch; new hparam `reward_norm: {none, minmax, whiten}`.
- **Advantage (`gae.py`):** add an MC mode ($A = \mathrm{RS}(r) - V$, returns = RS(r)); a `compute_mc_advantage` helper selected by `advantage_mode: {gae, mc}`. Keep the existing ± 5 advantage clip.
- **Actor (`optimize`):** the main new code — replace the ratio block with $w = \exp(\tau\,\mathrm{clip}(A, \pm\delta))$, $L_{\text{actor}} = \mathrm{mean}(w \cdot \|v_\theta - u_t\|^2)$. The adapter must expose the velocity v_θ and CFM target $u_t = \boldsymbol{x}_1 - \boldsymbol{x}_0$ (a `return_kwargs=["velocity","cfm_target"]`-style output). New hparam `gcw_temperature` (τ=1).
- **Regularizer:** already covered — `kl_type="v-based"`, `kl_beta`= $\alpha = 1.0$.
- **Warmup:** optional `warmup_uses_grae` (first k=500 steps use $w = \exp(\tau\,(\mathrm{RS}(r) - \mu_B)/\sigma_B)$) instead of our frozen-policy bootstrap.

Simplification: this mode needs *no* SDE rollout and *no* per-step log-prob storage — ODE sampling + random-t interpolation suffice, so it can be cheaper than our PPO path.

3.4 Explicit Critic Guidance — the diffusion backbone as its own value function (arXiv:2605.27736)

Unique contribution.

1. **Backbone-as-value-function on noisy latents.** Convert the pretrained diffusion model into a timestep-conditioned scalar critic $V_\phi(\boldsymbol{z}_t, t, y)$ with lightweight heads (UNet: patchify + learnable summary token + attention head; DiT: critic-attention branches at layers 7/15/23 with AdaLN time modulation), initialized from the actor. This is *state-aligned* (it scores the actual noisy latents, unlike pixel-space critics that decode an OOD $\hat{\boldsymbol{x}}_0$ proxy), and reported 2.6–4.3× cheaper than CLIP/BLIP/HPS critics at higher reward.
2. **Two simple stabilizers:** AdaLN timestep conditioning on the value head (essential in ablation) and a *short* value-pretraining warmup (more is worse: GenEval $0.54 \to 0.64$ at 15 it, then declines).
3. **Inference-time steering for free:** the PPO-trained critic's latent gradient $g_t = \nabla_{\boldsymbol{z}_t} V_\phi(\boldsymbol{z}_t, t, y)$ is injected into the sampler (DDIM: $\hat{\varepsilon} = \varepsilon - \eta\sqrt{1-\bar{\alpha}_t}\, g_t$; flow: $\hat{v} = v + \eta\, g_t$), requiring no extra training.
4. **Multi-reward via per-reward value heads** (shared backbone, one head per reward); separate heads beat a shared head and mitigate reward hacking.

Core method. *This is true PPO.* Standard GAE (Eqs. 7–8) with $\gamma{=}\lambda{=}1$ (pure Monte-Carlo over short terminal-reward trajectories), global advantage normalization (Eq. 6, clip $[-10, 10]$), value-target clip $[-5, 5]$, the clipped policy surrogate with a tiny ratio clip (10^{-5} for SD1.5, 10^{-4} for SD3.5M, same high-dim Gaussian reasoning as ours), and value pretraining (Eq. 2). KL-to-base is discussed and *discarded* (it only shifts the optimization timescale, not the hacking); mitigation is multi-reward + early stopping.

Relation to ours. This is the closest match: same MDP, terminal reward, terminal value 0, GAE with the same TD recursion, global advantage whitening, clipped policy ratio, value pretraining, advantage clamping, CFG-free, one sample per prompt. The decisive difference is **Axis 2**: their critic *is* the diffusion backbone (shared weights, text conditioning, AdaLN), whereas ours is an independent lightweight net conditioned on $(\boldsymbol{z}_t, t)$ only. They also add inference-time steering and per-reward heads, which we do not have. Our `ppo_analysis` already names this "diffusion-model-as-value" direction as the backbone+value-head critic we evaluated and deferred — this paper is direct evidence the deferred direction pays off.

Implementation in Flow-Factory. Because we are already PPO+GAE, this is the one paper we adopt by *extending*, not replacing, the trainer. The GAE code, dual-optimizer scaffolding, value warmup, advantage whitening, value clipping and checkpointing *all carry over unchanged*; only the critic body and a few features are new.

- **Backbone-critic mode (`critic_type: {independent, backbone}`).** In `_initialization`, for `backbone` build value heads on the adapter backbone (DiT: critic-attention branches at layers 7/15/23 with AdaLN; UNet: summary-token attention head), *initialized from the actor*; the `critic_optimizer` trains the head params (and optionally the backbone-as-critic copy). Swap `self.critic(latents, t, channel_dim)` in `_compute_old_values` and `optimize` for the backbone+head forward (passing prompt embeds). Memory caveat: ∼2× a forward per value query; reuse cached features where possible.
- **Inference-time steering.** A sampler hook computing $g_t = \nabla_{\boldsymbol{z}_t} V_\phi$ and injecting it (velocity: $v \mathrel{+}= \eta g_t$; noise: $\varepsilon \mathrel{-}= \eta\sqrt{1-\bar{\alpha}_t} g_t$); new `steer_eta`. Pure inference feature, but depends on the backbone critic.

- **Multi-reward heads.** Stop pre-summing in `_compute_terminal_rewards`; keep a per-reward vector $R^{(m)}$; M value heads; run `compute_gae` per reward; store `step_advantages`/`step_returns` per reward; sum the per-head value losses. Builds on our source-aware `advantage_processor.reward_weig`
- **Smaller tweaks:** AdaLN time conditioning on the (independent) critic head; optional text conditioning (the paper found time conditioning matters most); a paper-faithful preset `gae_lambda=1.0` and value-target clip $[-5, 5]$.

4 Side-by-side Comparison

Tables 2 and 3 place all five methods on the two axes and the secondary design choices. Read together with Figure 1.

Aspect	**Ours (PPO)**	**DiffAC**	**VARD**	**AC-Flow**
Actor mechanism	PPO clipped PG	1-step AC (REINFORCE / DDPG)	value backprop (DPG)	adv.-weighted CFM regr. (GCW)
Importance ratio / clip	yes ($\pm 10^{-4}$)	no	no	no
Credit assignment	GAE(γ, λ)	one-step value	dense value (MC)	MC advantage $r - V$
Critic target	GAE returns, clipped	MC $\to R$ (perturbed states)	MC $\to R$	MC $\to$ RS(r)
Reward normalization	advantage whitening	none	none	min–max shaping
Regularization to base	optional KL (v/x)	consistency KL (refit ref)	MSE-as-KL (Lemma 1)	W_2 velocity-L^2
Is it PPO?	**yes**	no	no	no

Table 2: Algorithmic structure (Axis 1). Explicit Critic shares our column (true PPO + GAE) and is therefore listed in Table 3 instead, to highlight the remaining differences.

5 Implications and a Roadmap for Flow-Factory

Where our trainer already sits. Flow-Factory's PPO is the *PPO-clip + GAE + independent-critic* point of the design space. It coincides with Explicit Critic on Axis 1 and is the cheap/model-agnostic extreme of Axis 2. The three non-PPO papers are best read as *alternative actor mechanisms* bolted onto the same value scaffolding we already own (rollout, terminal reward, $V(\boldsymbol{z}_t, t)$ critic, warmup, dual optimizers, KL plumbing).

What is reusable across all four (no new infra). Terminal-reward SDE-MDP; the $V(\boldsymbol{z}_t, t)$ critic with a time embedding; the *Monte-Carlo value target* (our GAE at γ=λ=1); critic warmup/re-warmup (= "value pretraining" in every paper); dual optimizers; the v-/x-space reference penalty (covers AC-Flow's W_2 and VARD's MSE-as-KL anchor); advantage clamping at ± 5 (matches AC-Flow's δ); last-k-step training.

Recommended ordering of new work.

1. **Backbone-critic + inference steering + multi-reward heads** (Explicit Critic, §3.4). Highest value and lowest risk: it *extends* our existing PPO+GAE trainer rather than replacing the actor, and the steering/multi-reward features are direct user wins. Already scaffolded by our GAE / dual-optimizer / checkpoint code; the only new piece is the backbone value head per adapter family.

Aspect	Ours (PPO)	Explicit Critic	DiffAC	VARD / AC-Flow
Critic body	independent light net (PMA)	diffusion backbone + value head(s)	backbone-shaped (mol.); T2I n/s	VARD: reward-model LoRA / AC-Flow: CNN+MLP
State source for V	on-policy SDE rollout	on-policy rollout	forward-perturbed	diff. rollout / ODE interp.
Sampler	SDE (log-probs)	SDE/DDIM (log-probs)	SDE (log-probs)	must be differentiable / ODE
Reward differentiable?	not required	not required	not required	VARD: needs diff. *sampler*; AC-Flow: not required
Value warmup	bootstrap + re-warmup	short pretrain (15 it best)	MC pretrain + refit	MC pretrain + online
Text conditioning of V	no	yes (backbone)	—	VARD: yes (c)
Inference-time steering	no	**yes** ($\nabla_z V$)	no	no
Multi-reward	single weighted sum	**per-reward heads**	single	single
γ, λ default	1.0, 0.95	1.0, 1.0	n/a (1-step)	n/a (MC)

Table 3: Critic body and systems choices (Axis 2 + secondary). Ours and Explicit Critic are the two PPO methods, separated only by the critic body and the two features it unlocks (steering, per-reward heads).

2. **MC-advantage + AWR/GCW actor mode** (AC-Flow, §3.3): a contained actor-loss mode that reuses the critic and reference penalty, attractive because it drops the SDE rollout / log-prob storage cost.

3. **Value-backprop actor mode** (VARD, §3.2). New differentiable-rollout infrastructure; strong for differentiable rewards, but needs a differentiable sampler and more memory.

4. **Perturbation-state critic + consistency KL** (DiffAC, §3.1). Most research-y; Option B (perturbation critic target only) is a low-cost way to harvest its variance-reduction benefit inside our PPO loop, and is the cheapest of all follow-ups (the forward-noising already exists in `trainers/awm.py`). Detailed plan in `docs/ppo/ppo_analysis` § "Follow-up: DiffAC-style perturbed-state critic training".

One-line summary. *All four papers and our trainer agree on the value function on noisy latents; they disagree on the actor. Adopt Explicit Critic by extension (it is our algorithm with a stronger critic body), and treat DiffAC / VARD / AC-Flow as optional alternative actor modes over the same critic scaffolding.*

References

[1] Xiangxin Zhou, Liang Wang, Yichi Zhou. Stabilizing Policy Gradients for Stochastic Differential Equations via Consistency with Perturbation Process (DiffAC). *ICML 2024*; arXiv:2403.04154. https://arxiv.org/abs/2403.04154

[2] Fengyuan Dai, Zifeng Zhuang, Yufei Huang, Siteng Huang, Bangyan Liao, Donglin Wang, Fajie Yuan. VARD: Efficient and Dense Fine-Tuning for Diffusion Models with Value-based RL. arXiv:2505.15791, 2025. https://arxiv.org/abs/2505.15791

[3] Jiajun Fan, Chaoran Cheng, Shuaike Shen, Xiangxin Zhou, Ge Liu. Fine-tuning Flow Matching Generative Models with Intermediate Feedback (AC-Flow). arXiv:2510.18072, 2025. https://arxiv.org/abs/2510.18072

[4] Zhengyang Liang, Qihang Zhang, Ceyuan Yang. Explicit Critic Guidance for Aligning Diffusion Models. arXiv:2605.27736, 2026. https://arxiv.org/abs/2605.27736